

UNIVERSITI TEKNOLOGI MARA

**SECURE UITM CREDIT EXEMPTION
MANAGEMENT SYSTEM USING OPTICAL
CHARACTER RECOGNITION AND
MULTIFACTOR AUTHENTICATION**

IZZAH KHAYREEN BINTI MAHAZAN

PROF. MADYA TS. DR. ALYA GEOGIANA BINTI BUJA

**BACHELOR OF COMPUTER SCIENCE (HONS.) NETCENTRIC
COMPUTING**

OCTOBER 2025

Chapter 4

4.1 Web Interface Design

4.1.1 Landing Page

4.1.2 Registration and Logging In Interface

4.1.3 Student Interface

4.1.4 Academic Advisor Interface

4.1.5 Coordinator Interface

4.1.6 Resource Person Interface

4.1.7 External Lecturer Interface

4.1.8 HEA Personnel Interface

4.2 Backend Processes

4.2.1 Two-Factor Authentication (2FA)

4.2.2 Web Scraping Operations

4.2.3 Google Cloud Vision API Integration

4.2.4 PDF Text Extraction

4.2.5 Token-Based Submission System

4.3 Testing

4.3.1 Functionality Testing

4.3.2 Usability Testing

4.3.3 User Acceptance Testing (UAT)

4.3.4 Security Testing

CHAPTER 4

ANALYSIS AND DISCUSSIONS

4.1 Web Interface Design

4.1.1 Landing Page

The UiTM Credit Exemption Management System (UiTMCES) greets users with a professional and motivating landing page, as shown in Figure 4.1. The page features a full-screen hero image of graduating UiTM students, establishing a clear institutional identity and aspirational goal. Prominently displayed is the system's value proposition, headlined "Your Fast Track to Graduation," followed by a brief description highlighting key features such as 'smart OCR' and an 'effortless digital system'. The interface provides clear navigation for both new and returning users with "Login" and "Register" buttons in the header, alongside a central "Apply Now" call-to-action button, guiding users toward the primary function of the system.

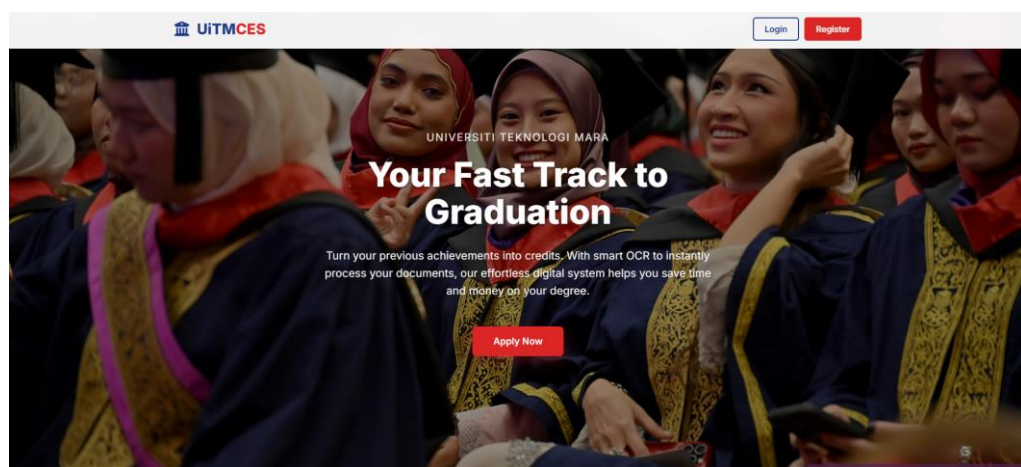


Figure 4.1 Landing page

4.1.2 Registration and Logging In Interface

The UiTM Credit Exemption Management System has been successfully developed with a user-friendly and functional interface. It features a well-crafted Create Account page designed to provide a secure and straightforward registration process for new users. The page, depicted in Figure 4.1, presents a clean modal form over a background of graduating students. The form requires users to input their Name, Email Address, and Phone Number, along with a role selection via the "Register as" dropdown. Users must also set and confirm a secure Password. The process is completed by clicking the prominent 'Create Account' button, ensuring a simple and secure user onboarding experience.

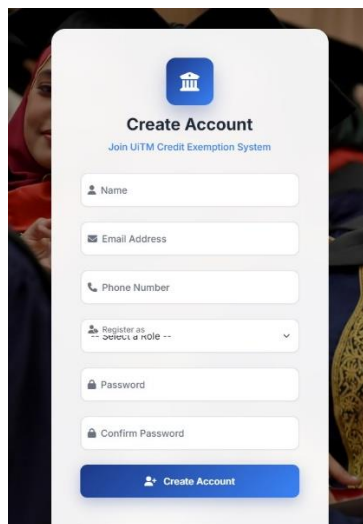


Figure 4.1 Create Account page

The UiTM Credit Exemption Management System also features a well-designed Login page that provides a secure and user-friendly experience for users to access their accounts. The Login page, depicted in Figure 4.2, presents a clean and straightforward interface. As shown, upon successful registration, a confirmation message is displayed instructing the user to "Please check your email to verify your account". The page includes essential input fields for Email Address and Password, a "Remember me" checkbox for convenience, a "Forgot your password?" link, and a clear "Sign In" button. Additionally, a link to the "Create Account" page is provided for new users.

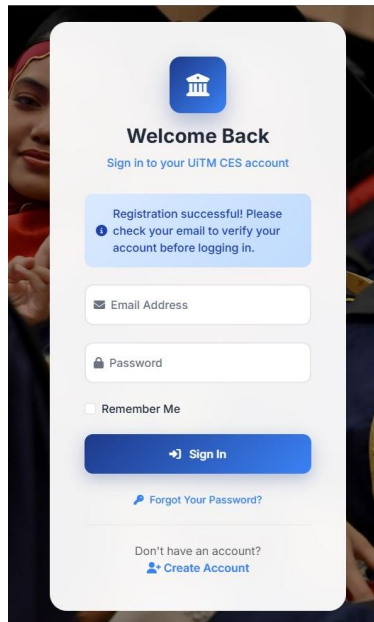


Figure 4.2 Login page

To reinforce the security of the UiTM Credit Exemption Management System, a mandatory Set up Google Authenticator page is presented to the user after their initial login. This page, depicted in Figure 4.3, is a critical step in the two-factor authentication (2FA) setup. The interface provides clear, numbered instructions: 1) Download the app, 2) Scan the QR code or enter the secret code, and 3) Enter the 6-digit code for verification. An important warning is displayed, noting that the setup is mandatory before continuing. This design choice significantly enhances account security by requiring a time-based one-time password (TOTP) for all subsequent logins.

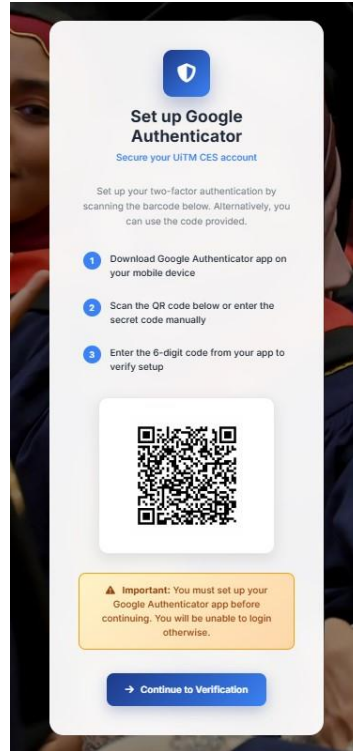


Figure 4.3 Google Authenticator Setup page

Once two-factor authentication is enabled, all subsequent login attempts will direct the user to the Verify 2FA page, as depicted in Figure 4.4. This page serves as the second layer of security. The interface clearly prompts the user to "Enter the code from your Google Authenticator app" and provides a single input field for the 6-Digit Code. A "Verify & Login" button is used to submit the code. The page also informs the user that the "Code refreshes every 30 seconds," reinforcing the time-based nature of the security. This seamless integration of 2FA ensures that user accounts remain secure without creating unnecessary friction for the user.

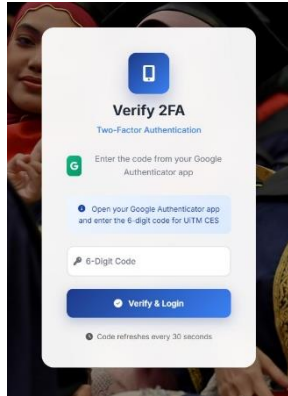


Figure 4.4 Verify 2FA page

4.1.3 Student Interface

Upon successful login, the user is directed to the main Student Dashboard, which serves as the central hub for all user activities within the system. The dashboard, depicted in Figure 4.3, features a clean, card-based layout and a personalized welcome message. The design ensures that users can log in efficiently and are presented with the most relevant information immediately. The interface highlights three "Quick Actions" for key tasks: New Application, Track Status, and Update Profile. Additionally, an "Important Information" section provides easy access to resources such as Terms & Conditions and the Academic Calendar. This design provides a streamlined and intuitive experience, allowing students to navigate to all essential functions of the UiTM Credit Exemption Management System from a single page.

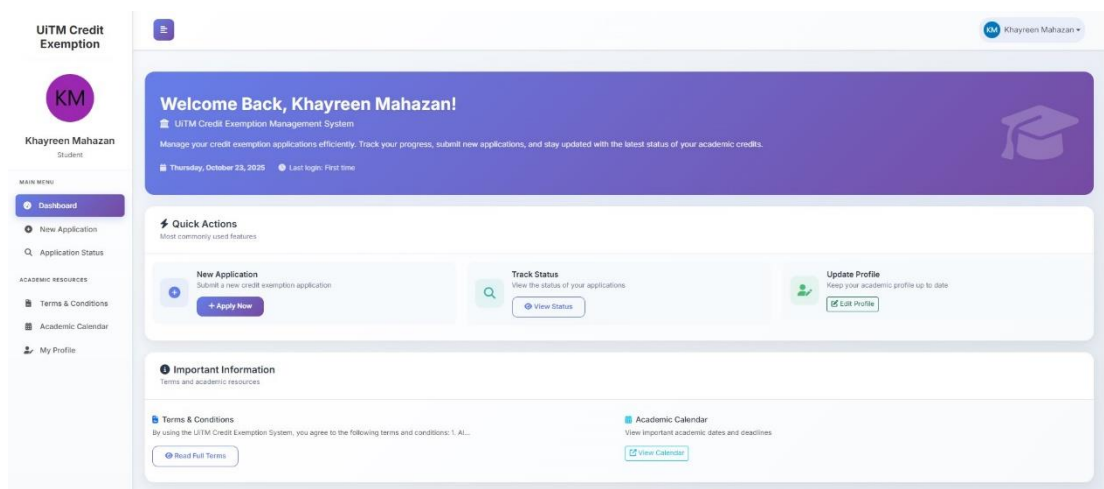


Figure 4.3 Student Dashboard page

The screenshot displays the 'New Credit Exemption Application' page. On the left, a sidebar identifies the user as 'Khayreen Mahazan' and provides navigation options: Dashboard, New Application (highlighted), Application Status, Terms & Conditions, Academic Calendar, and My Profile. The main content area is titled 'New Credit Exemption Application' and is divided into two steps.
Step 1: Fill in Your Details is organized into three sections:
1. **Student Information**: Includes fields for 'Full Name *', 'Student ID *', 'IC Number*' (with a placeholder '000000-00-0000'), and 'Home Address *'.
2. **Current UiTM Details**: Includes dropdown menus for 'Current Campus *' and 'Current Faculty *', text input fields for 'Programme Code *' and 'Mode of Study *', a dropdown for 'Current Semester *', and a radio button selection for 'Previous Institution (IPT)' with options 'UiTM (Previous Campus/Program)' and 'Non-UiTM Institution'.
3. **Step 2: Upload Transcript**: Features a 'Transcript (PDF only, Max 5MB) *' label, a 'Choose File' button (showing 'No file chosen'), and a note to 'Upload PDF of your official transcript'. A prominent purple button labeled 'Scan Transcript & Proceed' is located at the bottom right of the form area.

Figure 4.4 New Credit Exemption Application page

The UiTM Credit Exemption Management System (UiTM Credit Exemption Management System) includes a well-crafted New Credit Exemption Application page designed to facilitate a smooth and secure data collection process. The application page, depicted in Figure 4.4, features a clean and intuitive multi-step interface to guide the user.

The first step, "Step 1: Fill in Your Details," is organized into distinct sections to ensure all necessary data is captured. Under Student Information, users must provide their Full Name, Student ID, IC Number, and Home Address. Following this, the Current UiTM Details section requires the user to input their Current Campus, Current Faculty, Programme Code, Mode of Study, and Current Semester. Finally, the user must specify their Previous Institution (IPT) by selecting either "UiTM (Previous Campus/Program)" or "Non-UiTM Institution".

The process concludes with "Step 2: Upload Transcript," which provides a clear file input for the user to upload their official academic transcript. This structured design ensures that users can easily and accurately complete their application, maintaining a positive user experience and reinforcing the integrity of the data collected by the UiTM Credit Exemption Management System.

4.1.4 Academic Advisor Interface

The administrative side of the UiTM Credit Exemption Management System provides a robust interface for academic advisors, referred to as Academic Advisors, to manage and review student applications. Upon logging in, the academic advisor is presented with the Academic Advisor Dashboard, depicted in Figure 4.8. This dashboard provides an immediate, high-level overview of the application workload. The interface features four prominent summary cards: Applications Pending Review, Total Applications Reviewed, Courses Exempted by OCR, and Total OCR Processed Courses. Below this, a "Pending Applications" table lists all submissions requiring action, clearly organized by Submitted On, Student Name, Student ID, and Status, with a "View Details" button for each entry.

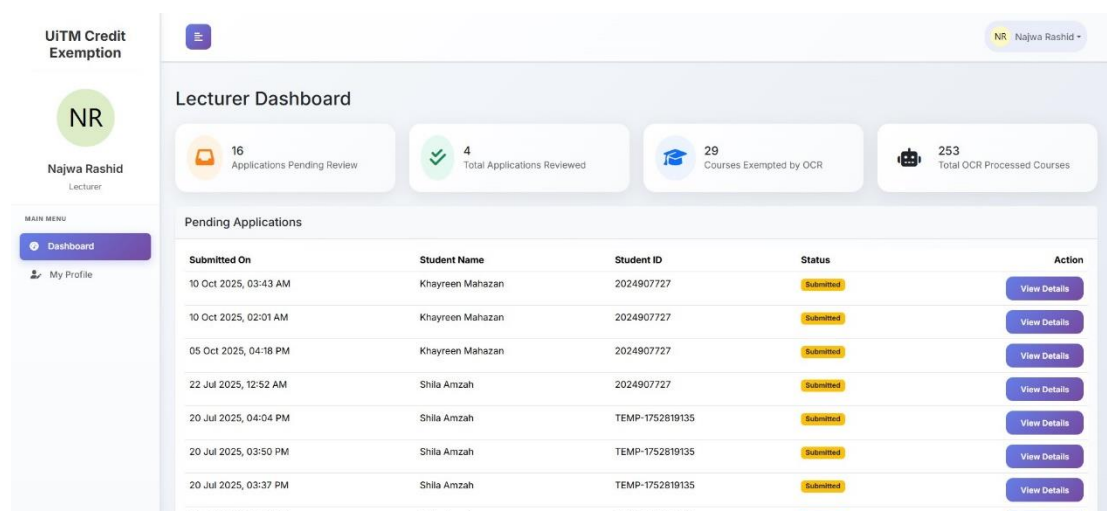


Figure 4.8 Academic Advisor Dashboard page

Upon selecting "View Details" for an application, the academic advisor is directed to the Application Details page, shown in Figure 4.9. This comprehensive page is designed to streamline the review process. It begins with basic Student Information (Student Name, Student ID, Diploma Institution, Diploma Program) and the current Application Status. A button to View Transcript (PDF) is also provided for manual verification.

The core of this interface is the OCR Processing Summary, which presents data-driven insights in summary cards, including "Courses Exempted," "Needs Review," "Program Code," and "Total Courses." The system then intelligently divides the application into two sections. The first, Pre-Qualified Courses (OCR

Approved), lists all courses that automatically met the predefined criteria (e.g., found in database, grade C or above, match > 80%). The second section, Courses Requiring Manual Review, flags courses that failed the OCR criteria, providing a specific Reason (e.g., "Course not found in... database") and an OCR Status (e.g., "Not Found"). For both sections, the academic advisor is provided with a clear set of Action buttons (Approve, Reject, Forward) for each individual course, allowing for granular and efficient decision-making.

UiTM Credit Exemption

NR
Najwa Rashid
Lecturer

MAIN MENU
Dashboard
My Profile

<< Back to Dashboard

Application Details - Khayreen Mahazan

Student Information

Student Name	Khayreen Mahazan	Student ID	2024907727
Diploma Institution	UiTM Melaka - Jasin	Diploma Program	DIPLOMA SAINS KOMPUTER
Application Status	Submitted		

Academic Transcript
View Transcript (PDF)

OCR Processing Summary

10 Courses Exempted	Needs Review	CSC251 Program Code	28 Total Courses
------------------------	--------------	------------------------	---------------------

Pre-Qualified Courses (OCR Approved)
These courses met all OCR criteria: Found in database + Grade C or above + Match >80%

Course Code	Course Name	Grade	Equivalent To	Match %	Action
CSC159	COMPUTER ORGANIZATION	A	CSC429	90%	☒ Approve ☒ Reject ☒ Forward
CSC159	INTERPRETATION OF ACCOUNTING	A	CSC429	80%	☒ Approve ☒ Reject ☒ Forward
MAT133	PRE CALCULUS	A	MAT406	88%	☒ Approve ☒ Reject ☒ Forward
CSC253	INTERACTIVE MULTIMEDIA	A	CSC493	88%	☒ Approve ☒ Reject ☒ Forward
CSC186	OBJECT ORIENTED	B	CSC432	88%	☒ Approve ☒ Reject ☒ Forward

Courses Requiring Manual Review
These courses did not meet OCR exemption criteria and require manual evaluation.

Course Code	Course Name	Grade	OCR Status	Reason	Action
MGT162	FUNDAMENTALS OF MANAGEMENT	A-	Not Found	Course not found in CS251 equivalency database	☒ Approve ☒ Reject ☒ Forward
CSC305	PROGRAMMING PARADIGMS	A-	Not Found	Course not found in CS251 equivalency database	☒ Approve ☒ Reject ☒ Forward
MAT210	DISCRETE MATHEMATICS	A	Not Found	Course not found in CS251 equivalency database	☒ Approve

Figure 4.9 Review Application page

4.1.5 Coordinator Interface

4.1.6 Resource Person Interface

4.1.7 External Lecturer Interface

4.1.8 HEA Personnel Interface

4.2 Backend Processes

This section elaborates on the major backend mechanisms that power the Secure UiTM Credit Exemption Management System. Each backend module focuses on a specific functionality such as user authentication, data acquisition, document processing, and secure collaboration. The following subsections present a detailed breakdown of these implementations.

4.2.1 Two-Factor Authentication (2FA)

The system implements two-factor authentication (2FA) using the Time-Based One-Time Password (TOTP) algorithm as specified in RFC 6238. This security layer adds an additional authentication factor beyond username-password combination, leveraging the Google Authenticator mobile application. According to Microsoft's security reports, multi-factor authentication can block 99.9% of automated attacks, making it essential for protecting sensitive academic records.

The 2FA system is implemented through the `LoginSecurityController` class using the PragmaRX\Google2FALaravel package. Key components include :

1. **Secret Key Generation:** 16-character Base32-encoded secrets with 80 bits of entropy
2. **QR Code Transmission:** Dynamic generation of scannable QR codes containing TOTP URI schemes
3. **TOTP Verification:** Time-based validation with ± 1 window (90 seconds total validity)
4. **Session Management:** Secure state transitions between password and 2FA phases
5. **Role-Based Redirection:** Intelligent routing to appropriate dashboards

The authentication follows a two-phase process. When credentials are submitted, the `LoginController` intercepts successful password authentication, stores the user ID in session, logs out the user, and redirects to 2FA verification.

```
// LoginController.php - Password authentication interception
protected function authenticated(Request $request, $user)
{
    $request->session()->put('2fa_user_id', $user->id);
    Auth::logout();
    return redirect()->route('2fa.index');
}
```

Figure 4.10 2FA flow from login to dashboard access

For first-time users, a cryptographic secret is generated and a QR code is created.

```
// LoginSecurityController.php - Secret generation
public function show2faForm(Request $request)
{
    $userId = $request->session()->get('2fa_user_id');
    if (!$userId) {
        return redirect('/login');
    }

    $user = User::findOrFail($userId);

    if (!$user->google2fa_secret) {
        $user->google2fa_secret = Google2FA::generateSecretKey();
        $user->save();

        $qrCodeUrl = Google2FA::getQRCodeInline(
            config('app.name'),
            $user->email,
            $user->google2fa_secret
        );

        return view('google2fa.index', [
            'qrCodeUrl' => $qrCodeUrl,
            'secret' => $user->google2fa_secret
        ]);
    }

    return view('google2fa.verify');
}
```

The QR code generation utilizes the `getQRCodeInline()` method, which returns a Base64-encoded PNG image embedded directly in the HTML response. This approach eliminates the need for external QR code generation services and ensures that the sensitive secret key never leaves the server environment except through the encrypted visual representation. The QR code follows the TOTP URI format:

`otpauth://totp/AppName:user@email.com?secret=SECRET&issuer=AppName`, which is the industry standard recognized by all TOTP-compatible authenticator applications.

TOTP Verification Algorithm

The verification process implements the Time-Based One-Time Password algorithm as defined in RFC 6238, which extends the HMAC-based One-Time Password (HOTP) algorithm specified in RFC 4226. The TOTP algorithm generates a six-digit code by computing an HMAC-SHA1 hash of the shared secret and a time counter, then extracting a dynamic truncation of the hash result.

The mathematical foundation of TOTP :

$$\text{TOTP} = \text{HOTP}(\text{Secret}, T)$$

where:

$$T = \text{floor}((\text{Current Unix Time} - T_0) / X)$$

$$T_0 = 0 \text{ (Unix epoch)}$$

$$X = 30 \text{ seconds (time step)}$$

$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA1}(K, C))$$

where:

K = shared secret key

C = counter (time-based)

The system implements verification through the `verify2fa()` method, which accepts the six-digit code submitted by the user and validates it against the stored secret using the current time window. The Google2FA library automatically handles time window tolerance, accepting codes from the previous, current, and next time windows (± 1 window = 90 seconds total validity) to account for clock skew between the server and the user's mobile device.

The 2FA implementation incorporates several security best practices to protect against various attack vectors such as :

Protection Against Brute Force Attacks: Although the current implementation validates codes server-side without explicit rate limiting, the 30-second time window and six-digit code space (1,000,000 possible combinations) create a natural barrier against brute force attempts. A potential attacker would have at most 30 seconds to try all combinations before the valid code changes, making brute force attacks computationally infeasible in practice.

Session Security: The system uses Laravel's encrypted session storage to protect the `'2fa_user_id'` value during the authentication transition. This prevents session fixation attacks where an attacker might attempt to inject a malicious user ID into the 2FA verification flow. The session is explicitly cleared of the `'2fa_user_id'` after successful verification, preventing reuse of the authentication token.

4.2.2 Web Scraping Operations

The system implements comprehensive web scraping functionality to populate the database with authoritative reference data from official Malaysian educational sources. This automated data acquisition strategy eliminates manual data entry, ensures data accuracy, and maintains synchronization with official institutional records. The scraping operations target two primary sources: the Malaysian Qualifications Agency (MQA) website and the Universiti Teknologi MARA (UiTM) official website, extracting over 700 data records across five distinct scraping commands.

Web scraping serves as a critical data collection methodology in scenarios where official APIs are unavailable or data export functionality is not provided. In the Malaysian higher education context, institutional data is publicly accessible through web interfaces but lacks programmatic access methods. The implementation uses HTTP client libraries to fetch HTML content and DOM

parsing techniques to extract structured data from unstructured web pages, transforming human-readable content into machine-readable database records.

Scraping Architecture

The web scraping infrastructure is implemented as Laravel Artisan console commands, allowing execution through the command-line interface with proper logging and error handling. This approach provides several advantages over controller-based scraping: commands can be scheduled through Laravel's task scheduler for periodic updates, they execute independently of HTTP request timeouts, and they integrate seamlessly with Laravel's service container for dependency injection.

```
// Base HTTP client configuration used across all scrapers
use GuzzleHttp\Client;

$client = new Client([
    'verify' => false,           // Disable SSL verification for MQA certificates
    'timeout' => 30,             // 30-second request timeout
    'headers' => [
        'User-Agent' => 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
    ]
]);
```

Figure 4.12 Guzzle client configuration

MQA Institutions Scraper

Extracts institutions from six MQA pages (IPTA, IPTS, Politeknik, Kolej Komuniti, Kolej Universiti, IPTS Kolej).

The MQA Institutions scraper extracts comprehensive listings of Malaysian educational institutions from six distinct MQA web pages, each representing a different institutional category: public universities (IPTA), private universities (IPTS), polytechnics, community colleges (Kolej Komuniti), university colleges

(Kolej Universiti), and private institution colleges. This multi-source scraping approach ensures complete coverage of the Malaysian higher education landscape.

The scraper processes each URL sequentially, fetching the HTML content and parsing table structures to extract institution names, states, and classifications. Data cleaning operations remove historical institution names (indicated by "Nama Lama" text), normalize whitespace, and filter duplicate entries. The extracted data is stored in the `institutions` table with fields for name, state, and type classification.

```
// ScrapeMqaInstitutions.php - Core scraping logic
public function handle()
{
    $urls = [
        'https://www2.mqa.gov.my/mqr/english/eAkrIPTA.cfm',
        'https://www2.mqa.gov.my/mqr/english/eAkrIPTS.cfm',
        'https://www2.mqa.gov.my/mqr/english/eAkrPoliteknik.cfm',
        'https://www2.mqa.gov.my/mqr/english/eAkrKolejKomuniti.cfm',
        'https://www2.mqa.gov.my/mqr/english/eAkrKolejUniversiti.cfm',
        'https://www2.mqa.gov.my/mqr/english/eAkrKolejIPTS.cfm'
    ];

    $client = new Client([
        'verify' => false,
        'timeout' => 30,
        'headers' => ['User-Agent' => 'Mozilla/5.0...']
    ]);
```

```
// Extract institution data from table rows
$crawler->filter('table tbody tr')->each(function (Crawler $node) use (&$institutions, $url) {
    $columns = $node->filter('td');
    if ($columns->count() >= 2) {
        $name = trim($columns->eq(0)->text());
        $state = trim($columns->eq(1)->text());

        // Clean institution names
        if (strpos($name, 'Nama Lama') !== false) {
            $name = trim(substr($name, 0, strpos($name, 'Nama Lama')));
        }

        $type = $this->determineType($url);

        $institutions[] = [
            'name' => $name,
            'state' => $state,
            'type' => $type
        ];
    }
});
```

Figure 4.13 `ScrapeMqaInstitutions.php` code highlighting URL array

UiTM Campuses Scraper

Extracts UiTM's 53+ campuses from MQA, generating unique campus codes.

28	Universiti Teknologi MARA (UiTM) Shah Alam	Selangor
29	Universiti Teknologi MARA Cawangan Johor Kampus Johor Bahru	Johor
30	Universiti Teknologi MARA Cawangan Johor Kampus Larkin	Johor
31	Universiti Teknologi MARA Cawangan Johor Kampus Pasir Gudang	Johor
32	Universiti Teknologi MARA Cawangan Johor Kampus Segamat	Johor
33	Universiti Teknologi MARA Cawangan Kedah Kampus Alor Setar	Kedah
34	Universiti Teknologi MARA Cawangan Kedah Kampus Sungai Petani	Kedah
35	Universiti Teknologi MARA Cawangan Kelantan Kampus Kota Bharu	Kelantan
36	Universiti Teknologi MARA Cawangan Kelantan Kampus Machang	Kelantan
37	Universiti Teknologi MARA Cawangan Melaka Kampus Alor Gajah	Malacca
38	Universiti Teknologi MARA Cawangan Melaka Kampus Bandaraya Melaka	Malacca
39	Universiti Teknologi MARA Cawangan Melaka Kampus Jasin	Malacca
40	Universiti Teknologi MARA Cawangan Negeri Sembilan Kampus Kuala Pilah	Negeri Sembilan
41	Universiti Teknologi MARA Cawangan Negeri Sembilan Kampus Kuala Pilah (Beting)	Negeri Sembilan
42	Universiti Teknologi MARA Cawangan Negeri Sembilan Kampus Rembau	Negeri Sembilan
43	Universiti Teknologi MARA Cawangan Negeri Sembilan Kampus Seremban	Negeri Sembilan
44	Universiti Teknologi MARA Cawangan Pahang Kampus Bukit Sekilau	Pahang
45	Universiti Teknologi MARA Cawangan Pahang Kampus Jengka	Pahang
46	Universiti Teknologi MARA Cawangan Pahang Kampus Kuantan	Pahang
47	Universiti Teknologi MARA Cawangan Pahang Kampus Raub	Pahang
48	Universiti Teknologi MARA Cawangan Perak Kampus Seri Iskandar	Perak
49	Universiti Teknologi MARA Cawangan Perak Kampus Tapah	Perak
50	Universiti Teknologi MARA Cawangan Perlis Kampus Arau	Perlis
51	Universiti Teknologi MARA Cawangan Pulau Pinang Kampus Bertam	Penang
52	Universiti Teknologi MARA Cawangan Pulau Pinang Kampus Bukit Mertajam	Penang
53	Universiti Teknologi MARA Cawangan Pulau Pinang Kampus Permatang Pauh	Penang
54	Universiti Teknologi MARA Cawangan Sabah Kampus Kota Kinabalu	Sabah
55	Universiti Teknologi MARA Cawangan Sabah Kampus Tawau	Sabah
56	Universiti Teknologi MARA Cawangan Sarawak Kampus Mukah	Sarawak
57	Universiti Teknologi MARA Cawangan Sarawak Kampus Mukah 2	Sarawak
58	Universiti Teknologi MARA Cawangan Sarawak Kampus Samarahan	Sarawak
59	Universiti Teknologi MARA Cawangan Sarawak Kampus Samarahan 2	Sarawak
60	Universiti Teknologi MARA Cawangan Selangor Kampus Dengkil	Selangor
61	Universiti Teknologi MARA Cawangan Selangor Kampus Puncak Alam	Selangor
62	Universiti Teknologi MARA Cawangan Selangor Kampus Puncak Perdana	Selangor
63	Universiti Teknologi MARA Cawangan Selangor Kampus Seksyen 17	Selangor
64	Universiti Teknologi MARA Cawangan Selangor Kampus Sungai Buloh	Selangor
65	Universiti Teknologi MARA Cawangan Terengganu Kampus Bukit Besi	Terengganu
66	Universiti Teknologi MARA Cawangan Terengganu Kampus Dungun	Terengganu
67	Universiti Teknologi MARA Cawangan Terengganu Kampus Kuala Terengganu	Terengganu
68	Universiti Teknologi MARA Cawangan Terengganu Kampus Kuala Terengganu (Cendering)	Terengganu

```
// Campus code generation
private function generateCampusCode($campusName, $state)
{
    $words = explode(' ', $campusName);
    $code = '';
    foreach ($words as $word) {
        if (strlen($word) > 0 && $word !== 'Kampus' && $word !== 'Cawangan') {
            $code .= strtoupper($word[0]);
        }
    }
    $code .= $this->getStateCode($state);
    return $code;
}
```

Figure 4.14 Campus code generation

UiTM Degree Programs Scraper

Extracts program information from `https://online.uitm.edu.my/degree.cfm`, distinguishing between faculty headers and program entries.

```
// Faculty and program extraction
$tables->each(function (Crawler $table) use (&$currentFaculty) {
    $table->filter('tr')->each(function (Crawler $row) use (&$currentFaculty) {
        $cells = $row->filter('td');
        if ($cells->count() >= 2) {
            $firstCell = trim($cells->eq(0)->text());

            if (strpos($firstCell, 'FAKULTI') !== false ||
                strpos($firstCell, 'AKADEMI') !== false ||
                strpos($firstCell, 'KOLEJ') !== false) {

                $currentFaculty = $this->extractFacultyInfo($firstCell);
                $this->storeFaculty($currentFaculty);

            } elseif ($this->isProgramCode($firstCell)) {
                $programCode = $firstCell;
                $programName = trim($cells->eq(1)->text());
                $this->storeProgram([
                    'code' => $programCode,
                    'name' => $programName,
                    'faculty_id' => $currentFaculty['id']
                ]);
            }
        }
    });
});

private function isProgramCode($text)
{
    return preg_match('/^[A-Z]{2,4}\d{3}$/', $text);
}
```

Code snippet showing the faculty/program extraction logic with state management. Program code validation uses regex pattern matching to identify valid UiTM program codes, which follow the format of 2-4 uppercase letters followed by 3 digits (e.g., CS251). This validation prevents false positives from extracting non-program text that happens to appear in the table structure.

```
private function isProgramCode($text)
{
    // Match UiTM program code format: 2-4 letters + 3 digits
    return preg_match('/^[A-Z]{2,4}\d{3}$/', $text);
}
```

Data Extracted:

1. 30 Faculty Records: 24 FAKULTI + 2 AKADEMI + 4 KOLEJ

2. 173+ Degree Programs: SARJANA MUDA programs across all faculties
3. Program Metadata: Program code, name, and faculty association

UiTM Diploma Programs Scraper

The diploma programs scraper follows a similar approach to the degree programs scraper but targets the `diploma.cfm` page and focuses exclusively on diploma-level programs. The implementation includes additional filtering to ensure only diploma programs are extracted, as the page may contain other program types or informational text.

Duplicate prevention is implemented through program code-based uniqueness checks, as some diploma programs may be offered by multiple faculties but should only appear once in the database. The scraper uses Laravel's `updateOrCreate()` method to handle idempotent operations, allowing the scraper to be run multiple times without creating duplicate records.

```
// ScrapeUiTMDiplomaPrograms.php - Duplicate prevention
Program::updateOrCreate(
    ['code' => $programCode], // Unique identifier
    [
        'name' => $programName,
        'type' => 'diploma',
        'updated_at' => now()
    ]
);
```

Production Results: 50+ unique diploma programs extracted with program codes, names, and associated faculty information.

Error Handling and Reliability

Web scraping operations face several potential failure scenarios that must be handled gracefully:

Network Failures: The Guzzle client is configured with a 30-second timeout to prevent indefinite hanging when servers are unresponsive. Network exceptions are caught, logged, and allow the scraper to continue with remaining URLs rather than terminating completely.

HTML Structure Changes: The most significant risk in web scraping is changes to the target website's HTML structure. The implementation uses flexible selectors that identify tables and rows by header content rather than fixed CSS classes or IDs, providing some resilience to layout changes. However, significant structural changes require scraper updates.

Character Encoding Issues: Malaysian institutional names contain Bahasa Melayu text with special characters. The scrapers preserve UTF-8 encoding throughout the extraction pipeline, from HTTP response decoding through database insertion.

SSL Certificate Problems: The MQA website uses SSL certificates that may not validate correctly in all environments. The `'verify => false'` configuration disables certificate verification, which is acceptable for public data scraping but should be noted as a security consideration.

4.2.3 Google Cloud Vision API Integration

The Google Cloud Vision API provides OCR capabilities for extracting text from student transcripts when direct PDF text extraction fails. This machine learning service accurately recognizes text from various document formats with confidence scoring for quality assurance.

```
// VisionServiceProvider.php - Service provider registration
namespace App\Providers;

use Google\Cloud\Vision\V1\ImageAnnotatorClient;
use Illuminate\Support\ServiceProvider;

class VisionServiceProvider extends ServiceProvider
{
    public function register()
    {
        $this->app->singleton(ImageAnnotatorClient::class, function ($app) {
            // Read credentials path from environment
            $credentialsPath = env(
                'GOOGLE_CLOUD_CREDENTIALS_PATH',
                storage_path('app/keys/service-account-credentials.json')
            );

            // Validate credentials file exists
            if (!file_exists($credentialsPath)) {
                throw new \RuntimeException(
                    "Google Cloud credentials file not found at: {$credentialsPath}"
                );
            }

            // Set environment variable for Google SDK
            putenv('GOOGLE_APPLICATION_CREDENTIALS=' . $credentialsPath);

            // Configure client with project ID
            $config = [];
            if (env('GOOGLE_CLOUD_PROJECT_ID')) {
                $config['projectId'] = env('GOOGLE_CLOUD_PROJECT_ID');
            }

            return new ImageAnnotatorClient($config);
        });
    }
}
```

'VisionServiceProvider.php' file highlighting the singleton registration and credential configuration.

Service Account Setup

The Google Cloud Vision API requires authentication through a service account, which is a special type of Google account representing an application rather than an end user. The service account setup process involves:

1. **Creating a Google Cloud Project:** Project ID 'uitm-credit-exemption-system' serves as the billing and resource management container.
2. **Enabling the Vision API:** Activation through the Google Cloud Console's API Library.
3. **Creating Service Account:** Generating a service account with "Cloud Vision API User" role.

4. **Generating Key File:** Downloading the JSON key file containing the private key and client email.
5. **Securing Key File:** Storing the key file in `storage/app/keys/` directory with restricted file permissions.

The service account JSON key structure contains:

```
{
  "type": "service_account",
  "project_id": "uitm-credit-exemption-system",
  "private_key_id": "abc123...",
  "private_key": "-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n",
  "client_email": "vision-api@uitm-credit-exemption-system.iam.gserviceaccount.com",
  "client_id": "123456789...",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token"
}
```

OCR Processing Implementation

The OCR processing logic resides in the `StudentApplicationController::store()` method as part of the transcript upload workflow. The implementation follows a try-catch pattern where PDF text extraction is attempted first, and Vision API is invoked only if direct extraction fails or produces empty results.

```
public function store(Request $request)
{
    $fullText = '';
    $extractionMethod = 'unknown';
    $ocrConfidence = 0;

    try {
        // Attempt PDF text extraction first
        $pdfParser = new Parser();
        $pdf = $pdfParser->parseFile(storage_path('app/private/' . $transcriptPath));
        $fullText = $pdf->getText();

        if (!empty(trim($fullText))) {
            $extractionMethod = 'pdf_parser';
            $ocrConfidence = 100;
        } else {
            throw new \Exception('PDF extraction returned empty');
        }
    }
}
```

Vision API invocation

```
// Store OCR metadata in transcript record
$transcript = Transcript::create([
    'id' => (string) Str::uuid(),
    'exemption_application_id' => $application->id,
    'file_path' => $transcriptPath,
    'file_hash' => hash_file('sha256', storage_path('app/private/' . $transcriptPath)),
    'digital_signature' => 'placeholder',
    'ocr_status' => 'completed',
    'ocr_processed_at' => now(),
    'ocr_raw_text' => $fullText,
    'ocr_confidence_score' => $ocrConfidence,
    'extraction_method' => $extractionMethod
]);
```

Database schema showing Transcript table with OCR metadata fields

Confidence Scoring and Quality Assurance

The Vision API provides confidence scores ranging from 0.0 to 1.0 for each detected text element, indicating the model's certainty in its recognition. The system calculates an average confidence percentage across all detected text blocks to produce a single quality metric for the entire transcript.

Confidence thresholds determine whether manual review is required:

Confidence Range	Interpretation	Action
90-100%	Excellent quality	Automatic processing
75-89%	Good quality	Automatic processing with flagging
50-74%	Moderate quality	Manual review recommended
Below 50%	Poor quality	Manual review required

The system stores the confidence score in the 'ApplicationSubject' records, allowing lecturers and coordinators to prioritize manual review of low-confidence extractions.

```
// Store confidence score per course
ApplicationSubject::create([
    'exemption_application_id' => $applicationId,
    'course_code' => $courseCode,
    'course_name' => $courseName,
    'grade' => $gradeGPA,
    'credit_hour' => $creditHours,
    'extraction_method' => 'ocr',
    'ocr_confidence_score' => $ocrConfidence,
    'needs_verification' => $ocrConfidence < 75
]);
```

Error Handling and Resilience

The Vision API integration implements multiple layers of error handling to ensure system resilience:

Network Failures: Guzzle HTTP client exceptions are caught and logged, with user-friendly error messages returned to the student.

API Quota Exhaustion: When the daily/monthly quota is exceeded, the API returns a 429 status code. The system logs this and notifies administrators while informing users that processing is temporarily unavailable.

Invalid Credentials: If the service account key is invalid or expired, authentication fails with a 401 error. The service provider's credential validation catches this during application bootstrap.

Empty Results: If the Vision API returns zero text annotations (possible for completely blank or corrupted images), the system logs this and marks the transcript for manual review.

Testing and Validation

The Vision API integration has been tested with various transcript formats:

Test Scenarios:

1. Native PDF transcripts (text-based): 100% fallback to PDF parser, Vision API not invoked.
2. Scanned PDF transcripts (image-based): Vision API successfully extracts text with 85-95% confidence.
3. Low-quality scans: Vision API extracts partial text with 50-75% confidence, flagged for manual review.
4. Handwritten annotations: Vision API detects some handwritten text but with low confidence.
5. Multi-column layouts: Vision API correctly preserves reading order in most cases.

4.2.4 PDF Text Extraction

PDF text extraction serves as the primary method for digitizing student transcript content, offering significant advantages over cloud-based OCR services in terms of processing speed, cost-effectiveness, and reliability. The implementation utilizes the Smalot PdfParser library, a pure PHP solution for extracting text from native PDF documents without external dependencies or network requests. This local processing approach achieves 100% accuracy for text-based PDFs, eliminates API costs, and provides instant results compared to the 2-5 second latency of the Google Vision API.

Implementation Architecture

The PDF extraction functionality is integrated within the `'StudentApplicationController::store()'` method as the first step of the transcript processing pipeline. The implementation follows a fail-fast pattern: attempt extraction, check for meaningful results, and proceed with course matching if successful; otherwise, initiate the fallback mechanism.

```

// Step 1: Attempt PDF text extraction
try {
    $pdfParser = new Parser();

    // Parse PDF file from storage path
    $pdf = $pdfParser->parseFile(
        storage_path('app/private/' . $transcriptPath)
    );

    // Extract all text content
    $fullText = $pdf->getText();

    // Validate extraction produced meaningful content
    if (!empty(trim($fullText))) {
        $extractionMethod = 'pdf_parser';
        $ocrConfidence = 100; // Native PDF = 100% accuracy

        Log::info('PDF text extraction successful', [
            'application_id' => $application->id,
            'file_path' => $transcriptPath,
            'text_length' => strlen($fullText),
            'method' => 'pdf_parser',
            'processing_time' => microtime(true) - $startTime
        ]);
    } else {
        // Empty result indicates scanned PDF
        throw new \Exception('PDF text extraction returned empty result');
    }
}

```

Code snippet showing the PDF extraction